

DSA Problem Ladder — Day 1

Monday — Easy: preserve order and define the output

September 14, 2026 | Priyanka | 120 minutes | C11 or Python 3

Central question: what does the processed part of the array promise?

1. Stable zero move. Move all zeroes to the end of an integer array in place, preserving the relative order of nonzero entries. Require $O(n)$ time and $O(1)$ auxiliary space. Example: $[0, 3, 0, -1, 3] \rightarrow [3, -1, 3, 0, 0]$.
2. Merge two sorted arrays. Return one sorted array containing every occurrence from both inputs. Inputs may be empty and contain duplicates; do not call a sorting function. Require $O(n+m)$ time; output storage is allowed.
3. Common values, once each. Given two sorted arrays, return their common values in increasing order without duplicates. Require $O(n+m)$ time and $O(1)$ auxiliary space excluding output. Example: $[1, 1, 2, 4]$ and $[1, 1, 3, 4, 4] \rightarrow [1, 4]$.
4. Sorted squares. Given sorted integers, return their squares in sorted order in $O(n)$ time, with output storage allowed. Example: $[-5, -2, 0, 3] \rightarrow [0, 4, 9, 25]$. Values have absolute value at most 10,000.

Session rules

- Minutes 0–10: closed-notes recall of one invariant and an edge case from the previous session (Monday: last week's boundary search).
- Minutes 10–110: solve today's queue in order. Check time after roughly 25 minutes per problem on easy days, 33 on medium days, and 50 on Wednesday. Unused time transfers forward; the queue is not a quota.
- Minutes 110–120: stop coding, record results, and explain one failed assumption and its correction. Mark unfinished work partial at the cutoff.
- Use C11 OR Python 3 consistently throughout the timed week. Translation is optional and does not count as another solve.
- Before coding, state a direct approach, its cost, and what property might remove work. After coding, give a correctness argument, time/space costs, and normal, boundary, and adversarial tests.
- After 15 minutes without a useful new step, write the blocker and choose to continue, request a small hint, or park the problem. Record hints; keep complete solutions closed during the attempt.

Count a solve only when working code meets the stated constraints, passes checks, and you can explain why it works. Separate independent and assisted solves. A correct slow baseline is partial if it misses the required bound. Testing time counts toward the session. If finished early, improve tests and proofs without increasing the solved count.

Attempt record

For each problem, submit its number, minutes spent, independent/assisted/partial/parked status, hints used, code, test results, invariant, and time/space analysis. Keep the same language throughout the timed week.

At 120 minutes record: attempted __; independently solved __; assisted solves __; partial __.

Exit ticket: Which assumption or boundary was hardest? Give one smallest failing example and explain your correction. Counts are evidence of this session, not a quota.